Electronics Research Institute

# Crop Yield Prediction Using a Decision Tree

Presented to Dr. Heba Draz, ECIP Internship Program Committee
hdraz@aucegypt.edu

Course: Hardware/Software Co-Design

Presented by: Ali Adel | ali.adel@email.com

https://eri.sci.eg

# Agenda

01 Recap

02 Code insights

03 Code Results

04 FPGA Flow

05 FSM Design

06 Floating Points

07 UVM Testbench Architecture

08 Conclusion

# Recap

What exactly did we talk about last week?

# Problem Statement & Objective

**Crop** yield depends on many interacting factors

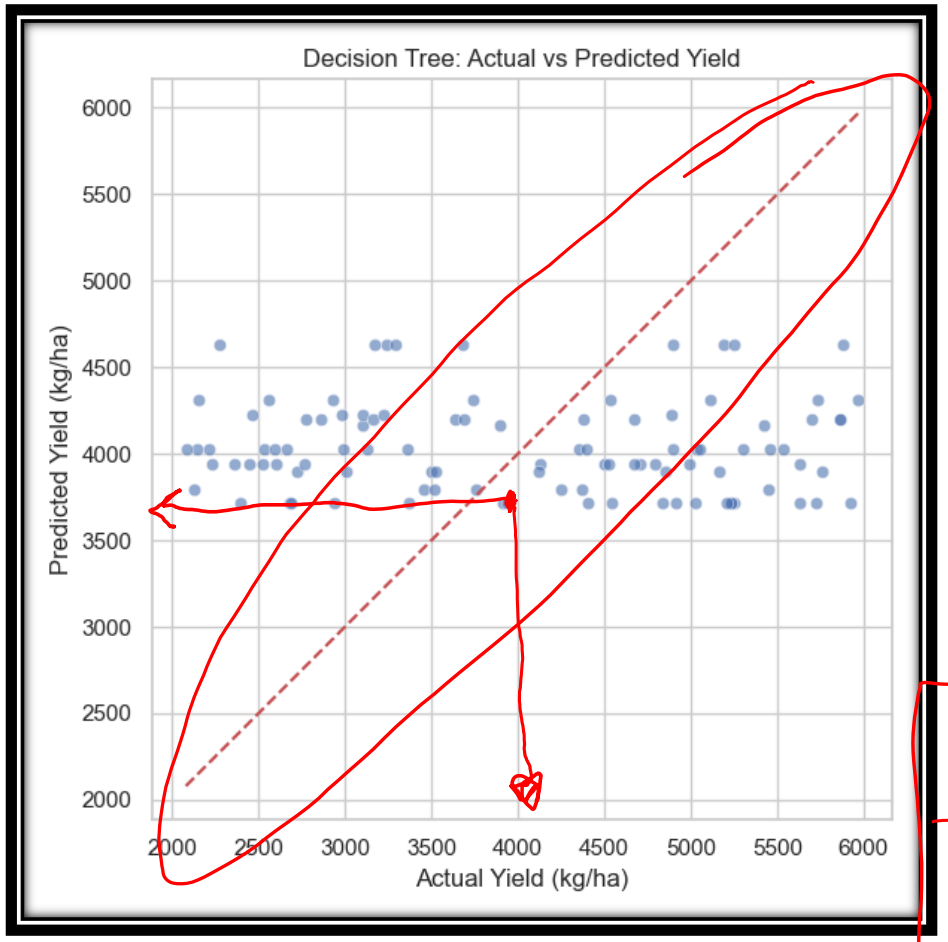
For example: soil moisture, temperature and sunlight hours.
Farmers usually predict crop yield using a mix of experience and observation.

## What about using magic?

- Benefits:
  - Faster and more precise than traditional guesswork.
  - Helps in planning harvests, marketing, and resource allocation.
  - Can adapt to climate change scenarios.

- Machine learning is the modern “magic” that turns data on weather, soil, and crops into accurate yield predictions.

# Results




3700

solu. try different models

→ SVM, KNN

# What is Next?

```
Root Split:
• Soil moisture ≤ 0.49
  ├─ Low sunlight → Lower yield (~3200–4700)
  └─ High sunlight → Yield depends on pH, rainfall, NDVI

• Soil moisture > 0.49
  ├─ Low NDVI → Low yield (~2900)
  └─ High NDVI → Yield depends on rainfall & temperature
```

python

# Code insights

Let’s take a close look at the code used.

# Code insights

- project_2
  - dataset_analysis.py
  - main.py
  - Makefile
  - output.log
  - model.py
  - requirements.txt

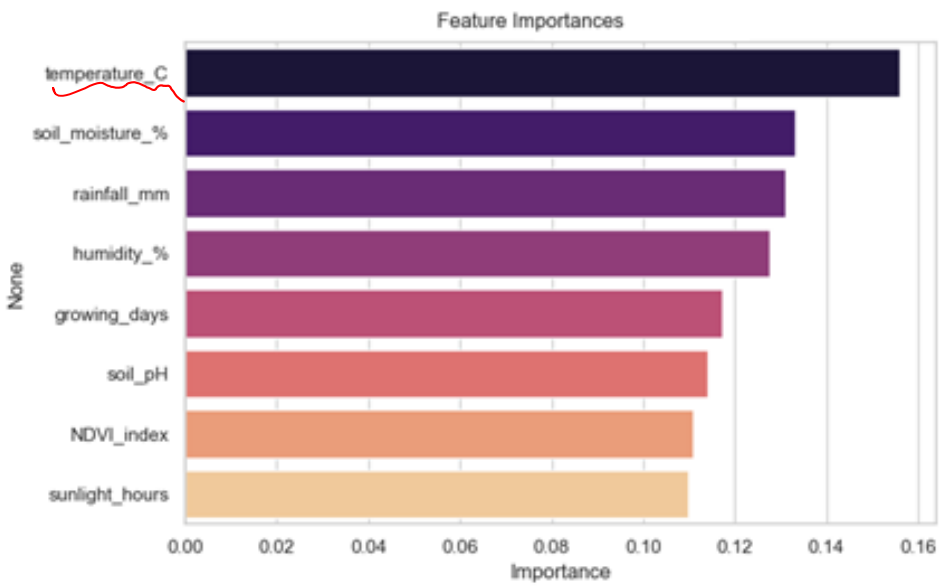
model.py

```python
df = pd.read_csv('datasets/Smart_Farming_Crop_Yield_2024.csv')

# Features
df['sowing_date']   = pd.to_datetime(df['sowing_date'], format='%Y-%m-%d')
df['harvest_date']  = pd.to_datetime(df['harvest_date'], format='%Y-%m-%d')
df['growing_days']  = (df['harvest_date'] -df['sowing_date']).dt.days

features = [
    'soil_moisture_%', 'soil_pH', 'temperature_C', 'rainfall_mm',
    'humidity_%', 'sunlight_hours', 'NDVI_index', 'growing_days'
]
```

# Code insights




model.py

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
def predict_tree_scaled(row_scaled):
    sm, ph, temp, rain, hum, sun, ndvi, gd = row_scaled

    if sm <= -0.032:
        if sun <= -0.811:
            if sm <= -0.061:
                if temp <= 0.291:
                    if rain <= 1.291:
                        return 4031.19 ...
```

# Code Results

- project_2
  - dataset_analysis.py
  - main.py
  - Makefile
  - output.log
  - model.py
  - requirements.txt

model.py

```
Prediction for First 7 Rows (Decision Tree Logic)
==================================================
Row   Actual Yield        Predicted Yield
--------------------------------------------------
1     4408.07             3792.80
2     5389.98             4629.53
3     2931.16             3715.58
4     4227.80             4023.42
5     4979.96             4313.71
6     4383.55             3792.80
7     4501.20             3792.80
==================================================
```

**model.py: Yield from the actual machine learning model (rules)**

- **Conclusion: The Python golden model is ready to be converted into an RTL design**

# FPGA Flow

Zero 20

## High-Level Design

Python code defines the intended behavior of the digital system

## Register-Transfer Level

RTL describes the hardware implementation with finite state machines and datapath

## Verification Environment

UVM testbench generates stimuli and monitors RTL output

## Functional Verification

Scoreboard compares RTL output against Python golden model

## Hardware Deployment

Verified RTL is synthesized and deployed onto an FPGA

③ model results ...

④ implementation I/O signals

⑤ Code

⑥ FPGA design

⑦ Co design

⑧ IP, using FPGA

model.py

RTL.v

9. SV

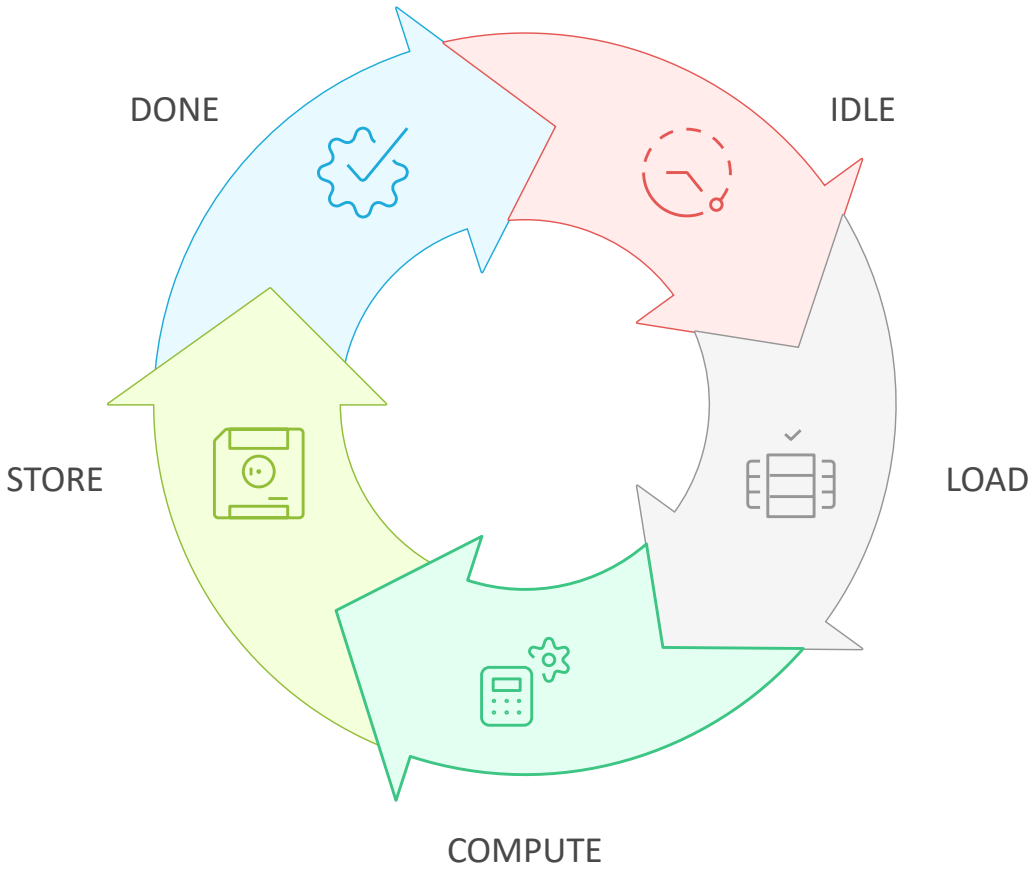
Dataset → model.py

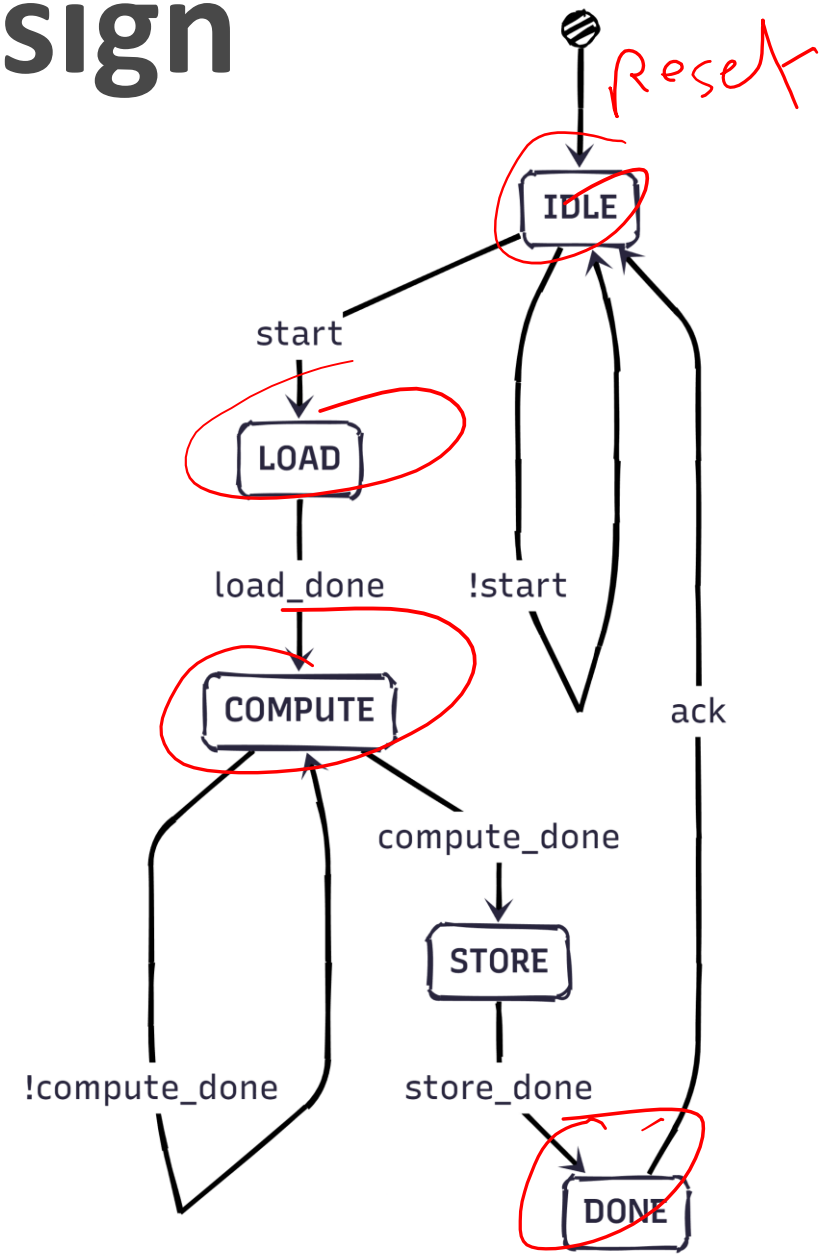
txt file

② Review.

① Intro, pb, soln. FPGA

# FSM Design


DONE
IDLE
LOAD
COMPUTE
STORE


Reset
IDLE
start
LOAD
load_done
!start
COMPUTE
ack
compute_done
STORE
!compute_done
store_done
DONE

# **Floating Points:** Scaling Numbers by 1000 for RTL

- Golden Model

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
def predict_tree_scaled(row_scaled):
    sm, ph, temp, rain, hum, sun, ndvi, gd = row_scaled

    if sm <= -0.032:
        if sun <= -0.811:
            if sm <= -0.061:
                if temp <= 0.291:
                    if rain <= 1.291:
                        return 4031.19 ...
```

- RTL Design

```verilog
if (sm <= -16'sd32) begin
    if (sun <= -16'sd811) begin
        if (sm <= -16'sd61) begin
            if (temp <= 16'sd291) begin
                if (rain <= 16'sd1291) begin
                    result = 32'sd4031190; // 4031.19 * 1000
```

4031.19 x 1000

4031190

- Convert floating-point numbers to integers for hardware-friendly operations

- Method:

$$\text{Scaled } \textit{Value} = \textit{Real Number} \times 1000$$

Q1.13

$2^{13}$

3,000

3.009

# UVM Testbench Architecture

**COVERAGE COLLECTOR**
Gathers test coverage metrics

**SCOREBOARD**
Compares DUT outputs with expected results

**MONITOR**
Observes the DUT's outputs

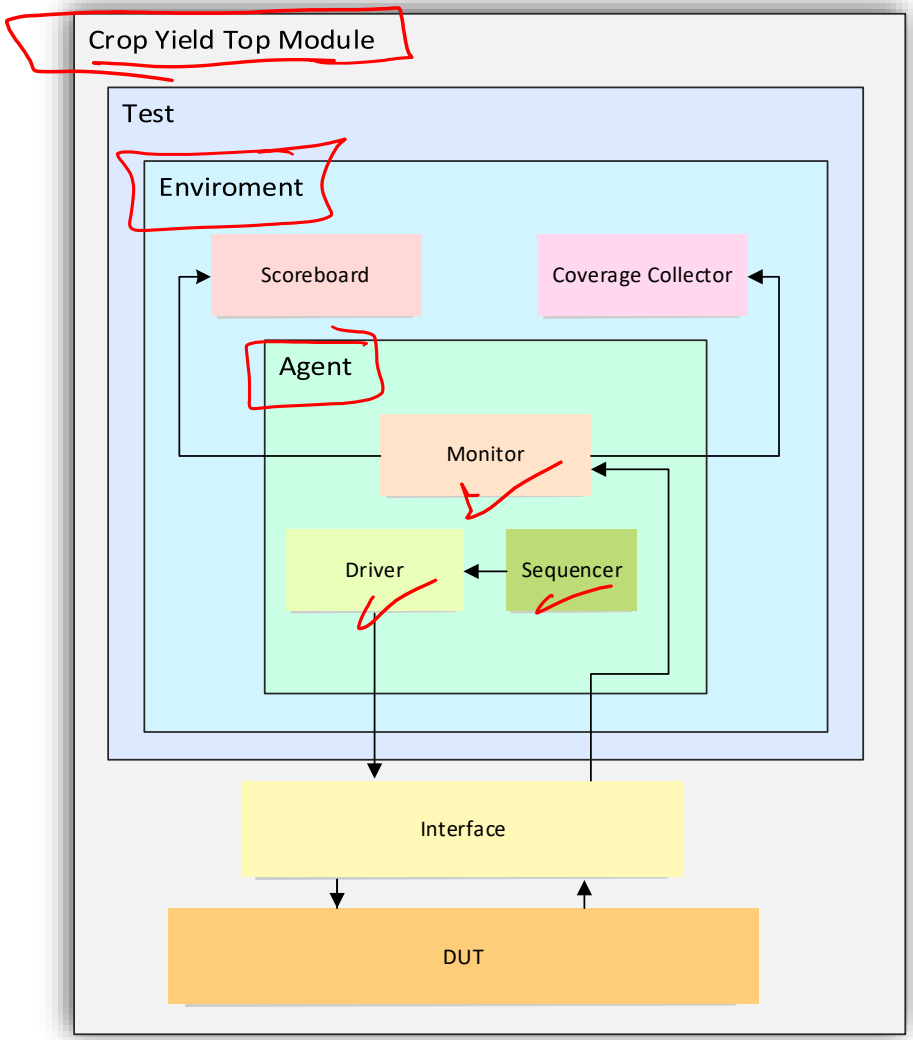
**INTERFACE**
Facilitates communication between components

**DRIVER**
Controls the DUT's inputs

**AGENT**
Represents a specific test scenario

**ENVIRONMENT**
Manages agents and their interactions

**TEST**
Contains all testbench components



# Conclusion

**STRENGTHS**

- Medium yield accuracy while using a simple algorithm DT
- Suitable for FPGA-based implementation

DT

**WEAKNESS**

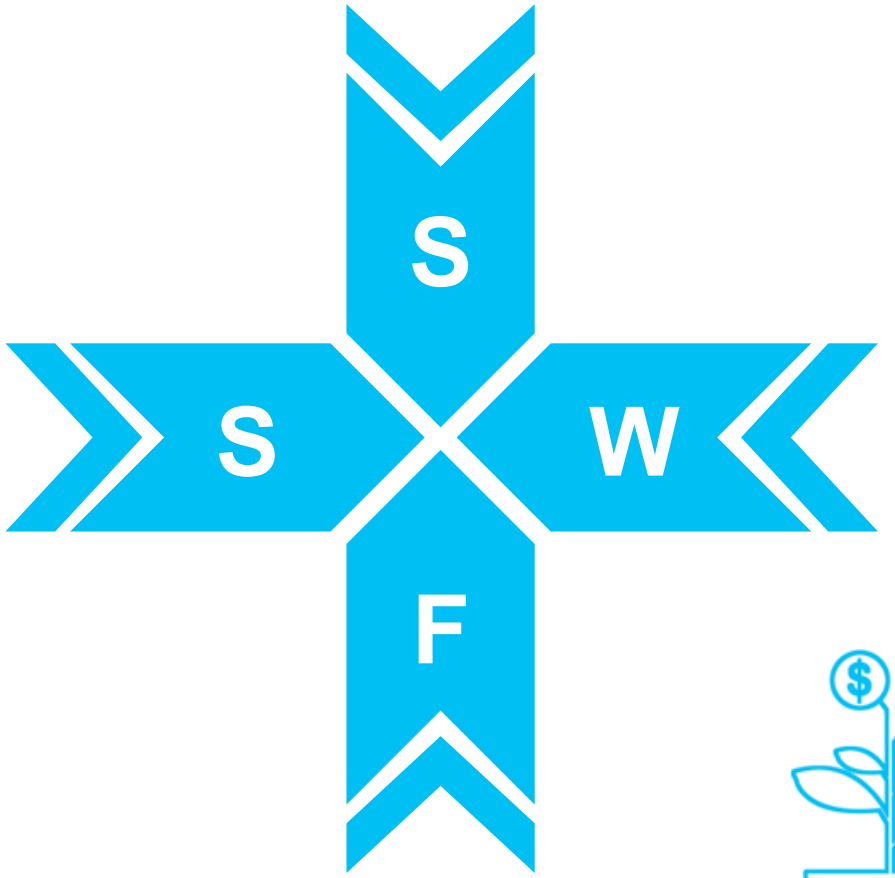
- Model performance depends strongly on data quality

**Future Work**

- Prepare and implement model on FPGA

**Summary**

- We finally reached the step where the golden model is ready!



# References

01 https://medium.com/@abhishekjainindore24/everything-about-random-forest-90c106d63989

02 https://mlkeww00jjj8.i.optimole.com/cb:_P4T.27e/w:380/h:422/q:mauto/f:best/https://www.displayr.com/wp-content/uploads/2018/07/what-is-a-decision-tree.png

03 Dataset:https://www.kaggle.com/datasets/atharvasoundankar/smart-farming-sensor-data-for-yield-prediction/data

04 Code:https://www.kaggle.com/code/adityaver/unit-5

05 https://en.wikipedia.org/wiki/Random_forest

# Thank you

Any Questions?